

Microdown Challenges

[Lien Github](#)

Réalisé par :

- LAHMER Rania Nafissa
- ATMANI Melissa
- OUARAB Cylia

2025-2026

Sommaire



1-Introduction

2-Compréhension du système existant

3-Les nouvelles fonctionnalités

4-L'objectif des nouvelles fonctionnalités

5-Conclusion

Introduction

Microdown = langage de balisage léger.



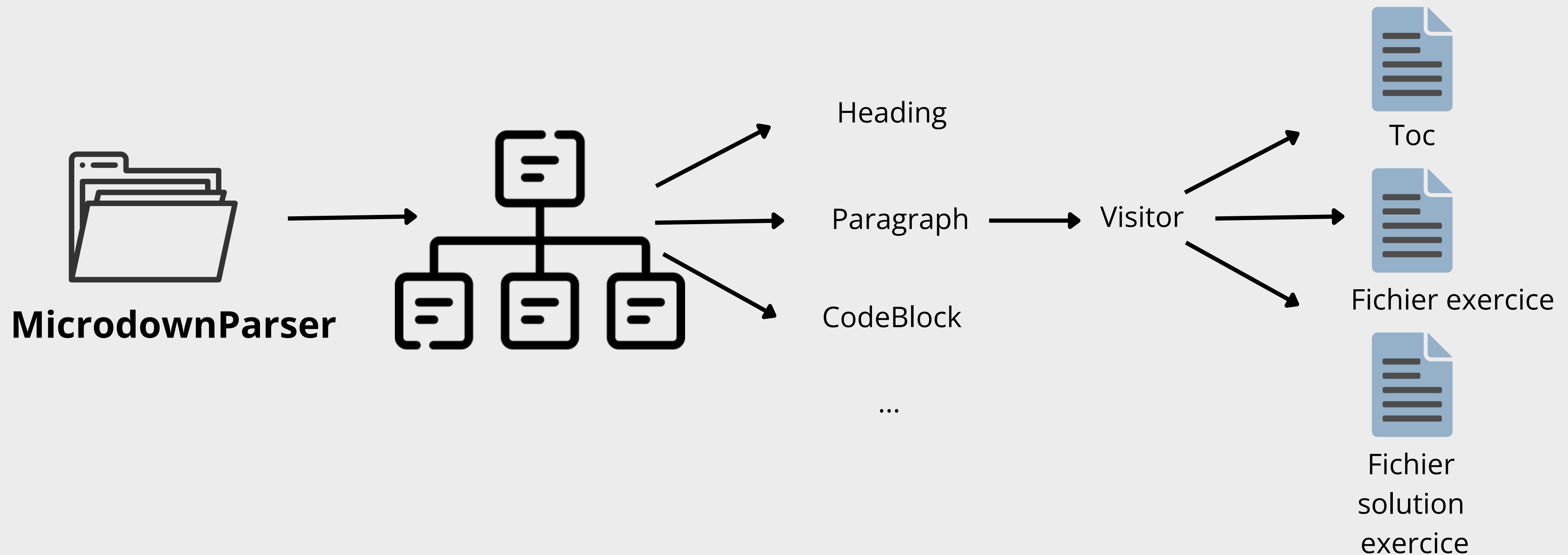
Sert pour :

- livres et PDF
- slides
- documentation
- sites statiques

Objectif du projet

L'objectif du projet était d'enrichir Microdown pour générer automatiquement une table des matières et gérer la séparation entre les énoncés d'exercices et leurs corrections.

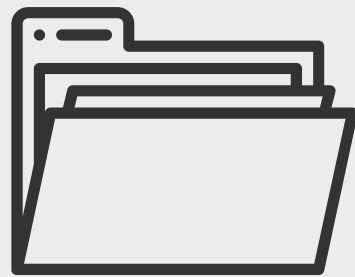
/04 Compréhension du système existant



/05 Compréhension du système existant



Problème du design actuel



MicrodownParser

Visitor



MicTextualBuilder

Aucun builder/visitor n'est dédié aux exercices

Les nouvelles fonctionnalités

- ⚙️ TOC: Producing Microdown for the TOC (SimpleTocGenerator)
- ⚙️ Microdown: Generating a plain text TOC(SimplePlainTextTocGenerator)
- ⚙️ TOC: Showing numbers(SimpleNumberTocGenerator)
- ⚙️ Microdown support for hiding corrections(MicrodownExerciceVisitor)

/07 Les nouvelles fonctionnalités



TOC: Producing Microdown for the TOC (SimpleTocGenerator)

- Contient le builder pour générer le texte.
- Filtre les titres selon showOnlyAbove.
- Définit la structure pour visiter un document (visit:) et un header (visitHeader:prefix:).
- Impossible à instancier directement (new génère une erreur) car c'est une classe abstraite.

SimpleMicrodownTocGenerator

- Hérite de SimpleTocGenerator.
- Extrait les titres de Microdown, utilisant des # selon le niveau du titre.

/08 Les nouvelles fonctionnalités

TOC: Producing Microdown for the TOC (SimpleTocGenerator)

The screenshot shows the Playground IDE interface. The code editor on the left contains the following Swift code:

```
1 | vis |
2 vis :=
  SimpleMicrodownTocGenerator new.
3 vis showOnlyAbove: 1.
4 vis visit: (Microdown parse: '
5 # Microdown
6 Bonjour tout le monde!
7 #Architecture
8 On a fini le projet.').
9 vis contents
10
```

The status bar at the bottom left indicates "Line: 10:1".

The right-hand pane shows the output of the code. The "Preview" tab is selected, displaying the generated Microdown content:

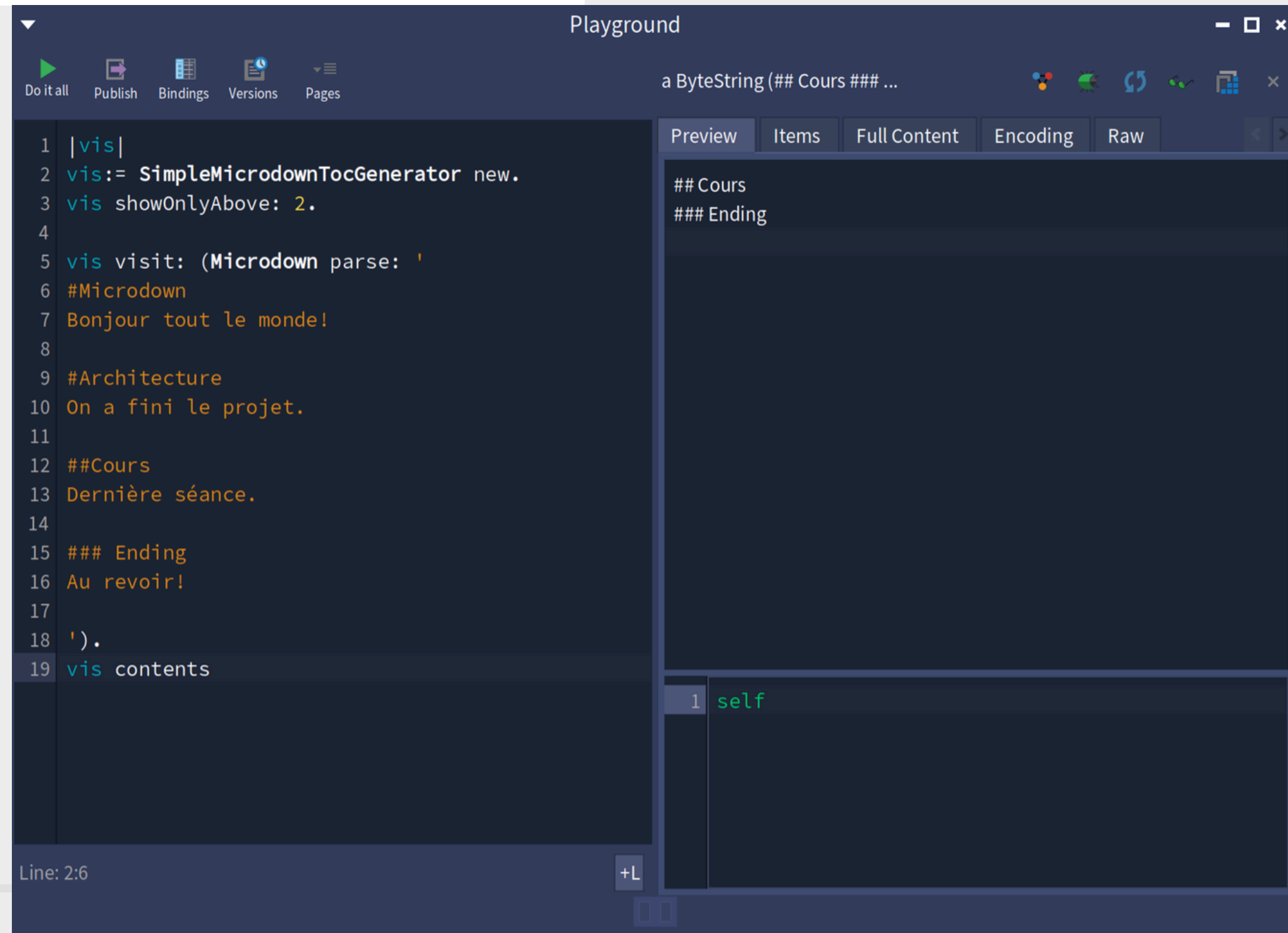
```
# Microdown
# Architecture
```

Below the preview, the "Items" tab shows a single item:

```
1 self
```


/09 Les nouvelles fonctionnalités

TOC: Producing Microdown for the TOC (SimpleTocGenerator)



The screenshot shows the Playground IDE interface. The left pane contains Swift code for a `SimpleMicrodownTocGenerator` class. The right pane shows the output of the code, which is a `ByteString` containing a table of contents (TOC) for a document. The TOC is formatted as a list of items with their corresponding page numbers.

```
1 |vis|
2 vis:= SimpleMicrodownTocGenerator new.
3 vis showOnlyAbove: 2.
4
5 vis visit: (Microdown parse: '
6 #Microdown
7 Bonjour tout le monde!
8
9 #Architecture
10 On a fini le projet.
11
12 ##Cours
13 Dernière séance.
14
15 ### Ending
16 Au revoir!
17
18 ').
19 vis contents
```

Output (a ByteString (## Cours ### ...)):

```
## Cours
### Ending
```

Line: 2:6

/10 *Les nouvelles fonctionnalités*

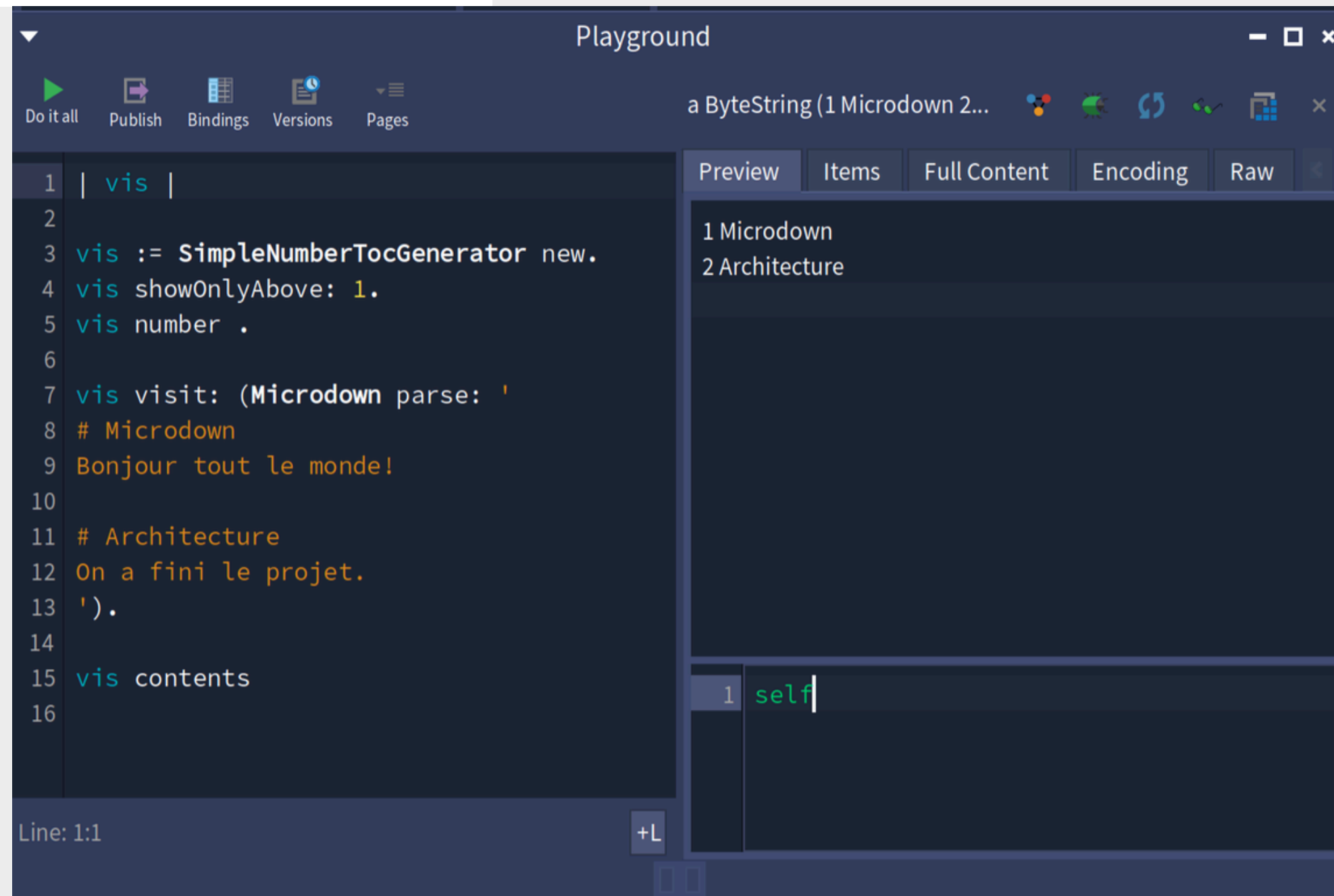


TOC: Showing
`numbers(SimpleNumberTocGenerator)`

- Hérite de SimpleTocGenerator.
- Extrait les titres de Microdown, utilisant des nombres selon la position du titre.
- Incrémenté un compteur à chaque titre rencontré(number).

/11 Les nouvelles fonctionnalités

TOC: Showing
numbers(SimpleNumberTocGenerator)



The screenshot shows the Scala Playground interface. The code editor on the left contains the following code:

```
1 | vis |
2
3 vis := SimpleNumberTocGenerator new.
4 vis showOnlyAbove: 1.
5 vis number .
6
7 vis visit: (Microdown parse: '
8 # Microdown
9 Bonjour tout le monde!
10
11 # Architecture
12 On a fini le projet.
13 ').
14
15 vis contents
16
```

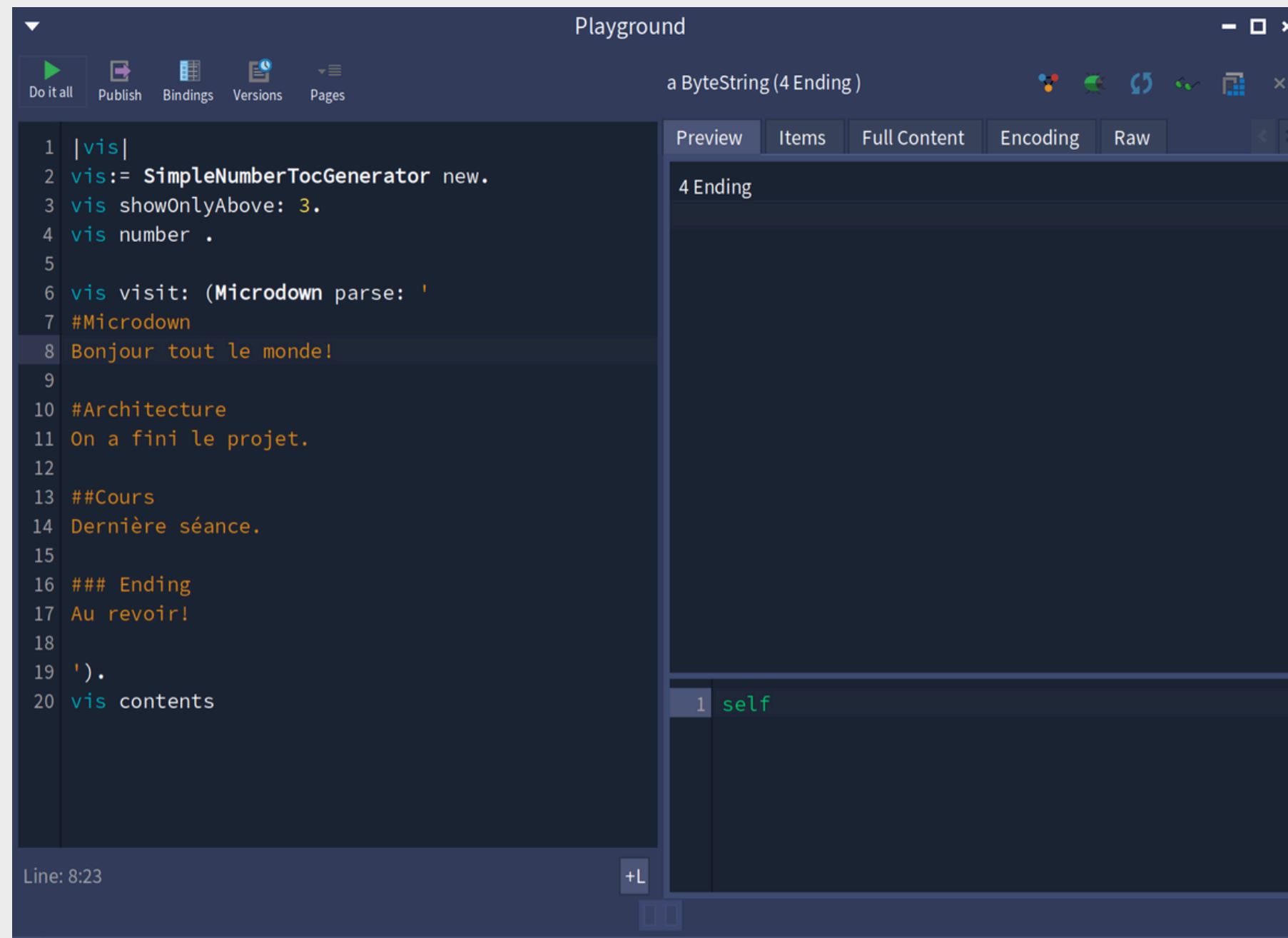
The preview pane on the right shows the output of the code, which is a Table of Contents (TOC) for a document. The TOC is displayed as a list of items with their corresponding line numbers:

```
1 Microdown
2 Architecture
```

The interface also includes a toolbar at the top with icons for "Do it all", "Publish", "Bindings", "Versions", and "Pages". The status bar at the bottom indicates "Line: 1:1".

/12 Les nouvelles fonctionnalités

TOC: Showing
numbers(SimpleNumberTocGenerator)



The screenshot shows a Swift Playground window titled "Playground". The code editor on the left contains the following Swift code:

```
1 |vis|
2 vis:= SimpleNumberTocGenerator new.
3 vis showOnlyAbove: 3.
4 vis number .
5
6 vis visit: (Microdown parse: '
7 #Microdown
8 Bonjour tout le monde!
9
10 #Architecture
11 On a fini le projet.
12
13 ##Cours
14 Dernière séance.
15
16 ### Ending
17 Au revoir!
18
19 ').
20 vis contents
```

The right pane shows the "Preview" tab with the output of the code, which is a rendered table of contents (TOC) for the provided Microdown text. The output is a list of sections with their corresponding line numbers and content.

Line: 8:23

/13 *Les nouvelles fonctionnalités*



**Microdown: Generating a plain text
TOC(SimplePlainTextTocGenerator)**

- Hérite de SimpleTocGenerator.
- Extrait des titres indentés selon le niveau des headers.

/14 Les nouvelles fonctionnalités

Microdown: Generating a plain text
TOC(SimplePlainTextTocGenerator)

The screenshot shows a Scala Playground interface. The left pane contains the following Scala code:

```
1 | vis |
2
3 vis := SimplePlainTextTocGenerator new.
4
5 vis visit: (Microdown parse: '
6 # Microdown
7 Bonjour tout le monde!
8
9 # Architecture
10 On a fini le projet.
11
12 #### Visitors
13 Un peu de texte
14
15 #### A builder
16 Fin du doc
17 ').
18
19 vis contents
20
```

The right pane shows the output of the code, which is a plain text TOC:

```
Microdown
Architecture
Visitors
A builder
```

Below the output, there is a section for the generated content, which is currently empty.

/15 *Les nouvelles fonctionnalités*

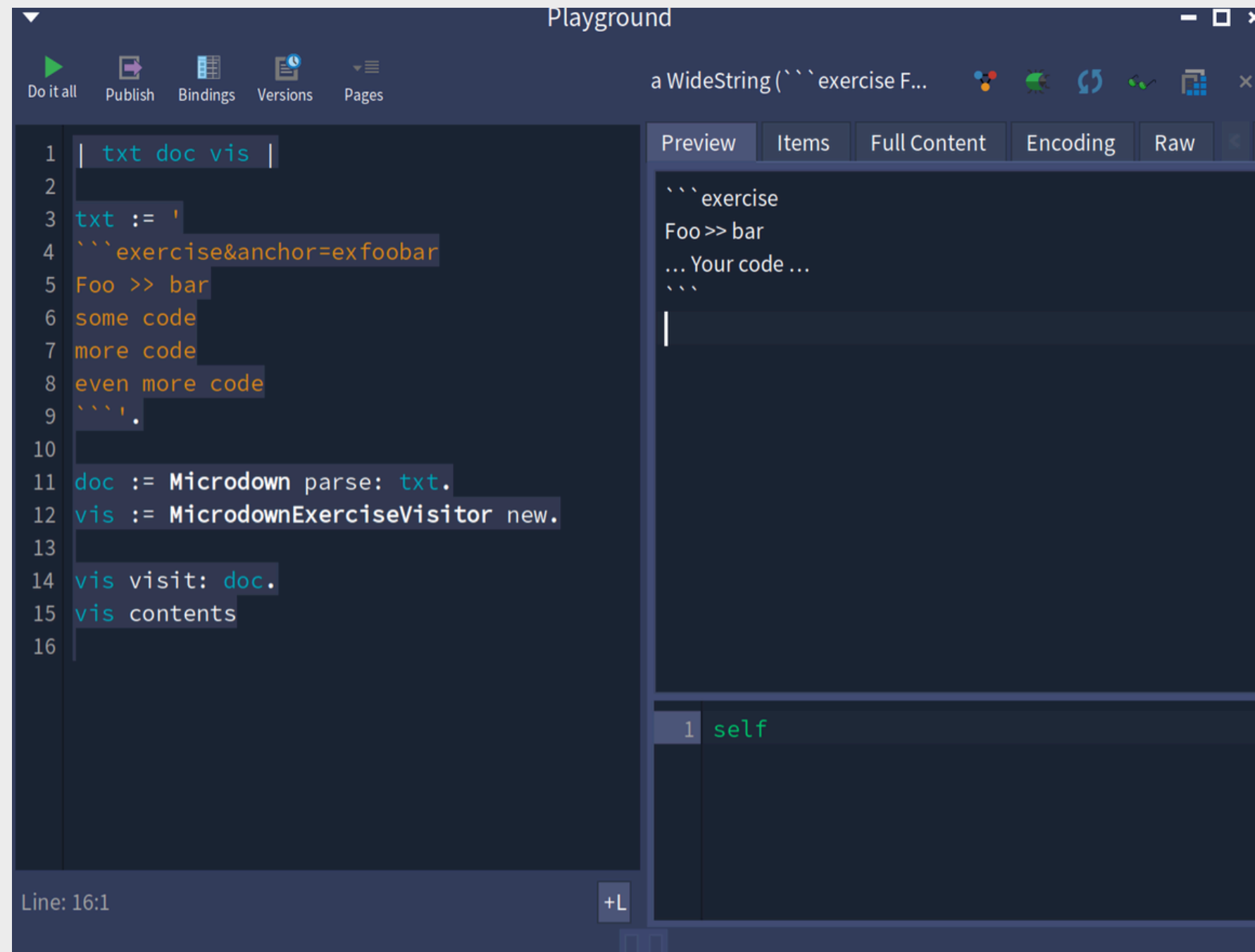


Microdown support for hiding corrections(MicrodownExerciceVisitor)

- Parcourt un document Microdown et gère les blocs de code d'exercices.
- Génère deux sorties :
 - 1- Le document principal où le code des exercices est remplacé par "Your code" afin de ne pas dévoiler la solution.
 - 2 - Le document de solutions contenant le code complet avec les références à l'ancre(identifiant unique) et au chapitre.

/16 Les nouvelles fonctionnalités

Microdown support for hiding
corrections(MicrodownExerciseVisitor)



The screenshot shows the Playground IDE interface. The main editor on the left contains the following code:

```
1 | txt doc vis |
2
3 txt := '
4 ```exercise&anchor=exfoobar
5 Foo >> bar
6 some code
7 more code
8 even more code
9 ```
10
11 doc := Microdown parse: txt.
12 vis := MicrodownExerciseVisitor new.
13
14 vis visit: doc.
15 vis contents
16
```

The status bar at the bottom left indicates "Line: 16:1".

On the right, there is a preview pane with tabs for "Preview", "Items", "Full Content", "Encoding", and "Raw". The "Preview" tab is selected, showing the rendered output of the code:

```
```exercise
Foo >> bar
... Your code ...
```
```

Below the preview pane, there is a small code editor showing the first line of the preview's internal code:

```
1 self
```


/17 Les nouvelles fonctionnalités

Microdown support for hiding corrections(MicrodownExerciseVisitor)

The screenshot displays a development environment with three main components:

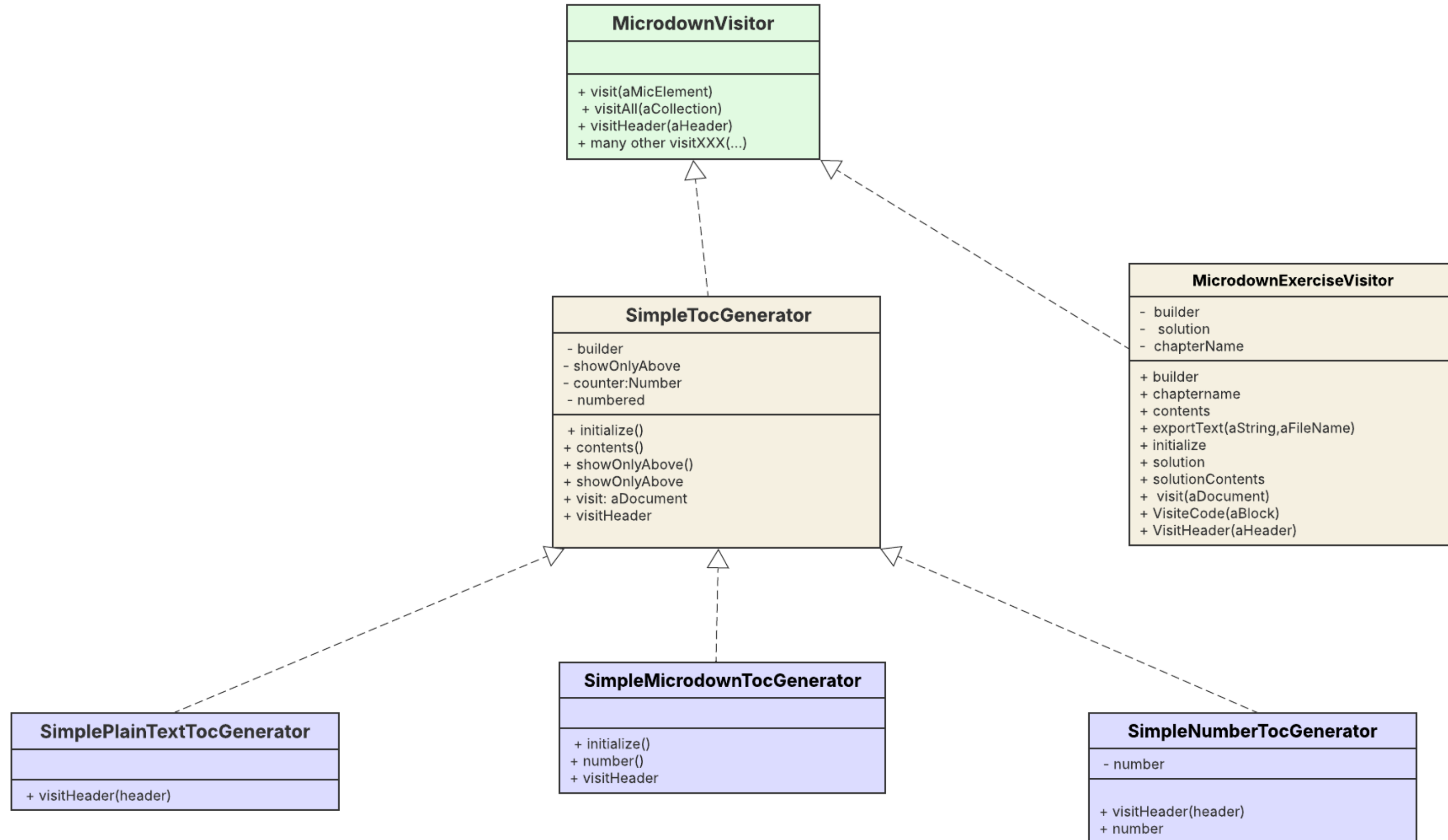
- Code Editor (Left):** Contains a script for a `StThreadSafeTranscript` object. The script defines a `txt` variable with a string containing exercise instructions, creates a `MicrodownExerciseVisitor` instance, and uses it to parse the transcript. The transcript content is: `txt := 'exercise&anchor=exfoobar\nFoo >> bar\nsome code\nmore code\neven more code\n'.`
- Variable Inspector (Middle):** Shows the state of the `StThreadSafeTranscript` object. It lists variables like `self`, `dependents`, `announcer`, `mutex`, `recordings`, `index`, `maxEntry`, and `worker` with their corresponding values.
- Markdown Editor (Right):** Shows the output of the `MicrodownExerciseVisitor` in a markdown file named `exerciseSolutions.md`. The output is a formatted solution for the exercise, including the instructions and the code snippets.

```
1 | txt doc vis |
2
3 txt := '
4 ``exercise&anchor=exfoobar
5 Foo >> bar
6 some code
7 more code
8 even more code
9 ``'.
10
11 doc := Microdown parse: txt.
12
13 vis := MicrodownExerciseVisitor new.
14 vis chaptername: 'Chap1'.
15
16 vis visit: doc.
17
18 Transcript show: vis contents; cr.
19
```

exerciseSolutions.md

```
1 Here is the solution of the code
2 @exercise@ in Chapter @Chap1@
3 ``
4 Foo >> bar
5 some code
6 more code
7 even more code
8 ``
```

/18 Le design après



/19 Tests

The screenshot displays the Microdown IDE interface. On the left, a project tree shows the file structure, with 'MicrodownToc-Tests' selected. The main editor area is divided into three panes: a class hierarchy pane on the left, a central pane showing the 'TestGeneratesSimpleToc' class with its methods 'TestImplementNumberTocGenerator' and 'TestSimplePlainTextTocGenerator', and a right pane showing the 'instance side' with methods 'testGeneratesExactSimpleToc' and 'testGeneratesExactSimpleTocAccordingToLevel'. At the bottom, a test results pane shows the execution status of three tests, all of which passed. The test results are as follows:

| Test Name | Results |
|---------------------------------|--|
| TestGeneratesSimpleToc | 2 ran, 2 passed, 0 skipped, 0 expected failures, 0 failures, 0 errors, 0 passed unexpected |
| TestImplementNumberTocGenerator | 2 ran, 2 passed, 0 skipped, 0 expected failures, 0 failures, 0 errors, 0 passed unexpected |
| TestSimplePlainTextTocGenerator | 2 ran, 2 passed, 0 skipped, 0 expected failures, 0 failures, 0 errors, 0 passed unexpected |

The bottom of the interface includes a toolbar with options for 'Flat', 'Hier.', 'Inst. side', 'Class side', 'Methods', 'Vars', and 'Class refs.', along with buttons for 'Comment', 'setUp', and 'Inst. side meth'.

/20 L'objectif des nouvelles fonctionnalités

- ✓ **Abstraction et extensibilité**
Utiliser le Visitor pour traiter tous les nœuds du document (headers, blocs de code, etc.) sans modifier la structure de Microdown.
- ✓ **Polymorphisme pour le traitement des nœuds**
 - **Redéfinir visitCode:** pour traiter différemment les blocs de code selon leur type (exercice ou autre).
 - **Redéfinir visitHeader:** pour récupérer dynamiquement le nom du chapitre.



Nous avons réussi à étendre le système existant en ajoutant de nouvelles fonctionnalités comme :

- la génération de Table of Contents (TOC)
- l'ajout de numérotation des titres
- la génération de TOC en texte brute
- la gestion des exercices avec génération automatique des fichiers de correction

/22

Merci !